WNDRVR
SUPPORT NETWORK

VXWORKS 7 MIGRATING AN RTP PROJECT INTO A LAYER TUTORIAL, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Migrating an RTP Project into a Layer Tutorial, SR0650

12 June 2020

1. MIGRATING AN RTP PROJECT INTO A LAYER

This tutorial demonstrates the workflow required to migrate an existing RTP project into a layer.

For this tutorial the workflow is as follows:

1. Create a custom layer.
2. Migrate an RTP Project into a VxWorks®custom layer.
3. Test the VSB and VIP containing the RTP layer.

WNDRVR

2. EXAMPLE: CREATING AN INSIDE CUSTOM LAYER AND INTEGRATING IT INTO A VSB

You can create a custom layer within the VxWorks source tree and integrate it into your VxWorks source build (VSB) project using the project build system.

Before You Begin

This example assumes you have access to the **cobble** downloadable kernel module (DKM) example project at *installDir* **\vxworks-7\samples\dkm**.

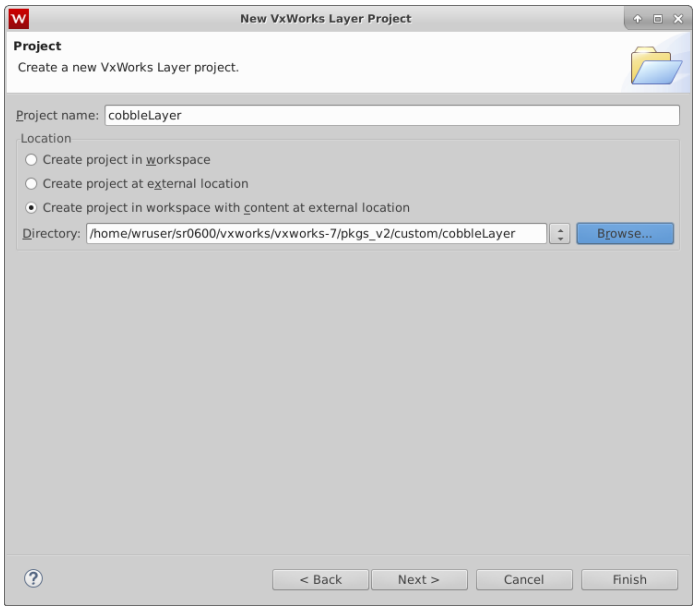
It also assumes you have a pre-existing VxWorks source build (VSB) and VxWorks image project (VIP) present in Workbench. Follow the steps below using a VxWorks simulator (VxSim) instance as the target.

About This Task

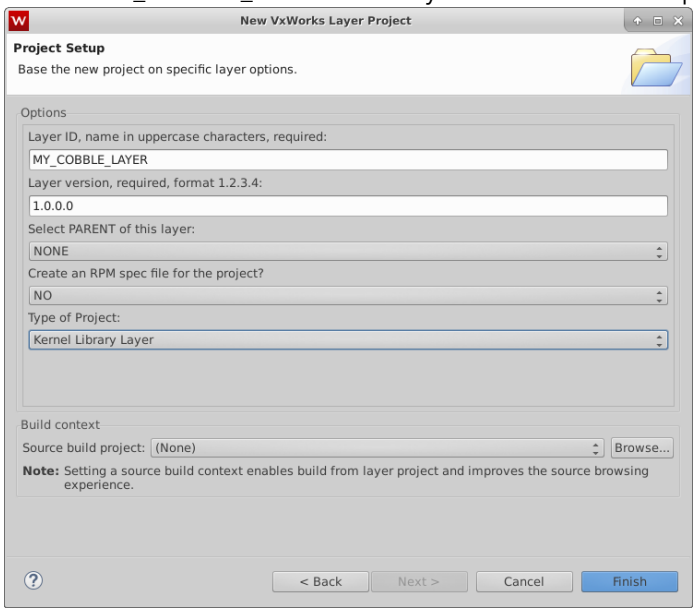
For this tutorial, you use the Workbench **cobble** example DKM project to represent the application code you want to integrate into a layer. Create a layer project, insert the **cobble** DKM application code into it, rebuild the VSB and VIP, and check that the target boots and runs.

Procedure

1. Create a folder for your new layer projects.
    1. Create a root directory within the Wind River installation at *installDir* **\vxworks-7\pkgs_v2\custom**.
    2. Create a folder for your new custom cobble layer project at *installDir* **\vxworks-7\pkgs_v2\custom\cobbleLayer**.
2. Create a VxWorks layer project in your new layer project folder and call it **cobbleLayer**.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. Select the option VxWorks 7 as the target operating system from the drop-down menu if you are prompted to choose the **Target Operating System**.
    3. From the Build type drop-down list, select **Layer**. Click **Next**.
    4. Give the Project the name **cobbleLayer**.
    5. Under the **Location** dialog, select **Create project in workspace with content at external location** and browse to the layer folder. Click **Next**.



6. Enter **MY_COBBLE_LAYER** as the layer ID. Leave all other options as default and click **Finish**.



3. Populate **cobbleLayer** with **cobble.c**.
    1. Go to *installDir* **\vxworks-7\samples\dkm\cobble**.
    2. Right-click on the **cobble.c** source file and select **Copy**.
    3. In the project explorer, navigate to **cobbleLayer**. Highlight the **src** folder within the **cobbleLayer** project and click **Paste**.
4. Update the **cobbleLayer** metadata file.
    1. Locate the file **layer.vsbl** at the root of the **cobbleLayer** project. Open the file for editing.
    2. Locate the line beginning with the tag **SYNOPSIS**. Replace the remaining text on this line with an appropriate description. For example, "This layer implements Cobble functionality".

```
Layer MY_COBBLE_LAYER {
SYNOPSIS This layer implements Cobble functionality
VERSION 1.0.0.0
```

    3. Save the file.

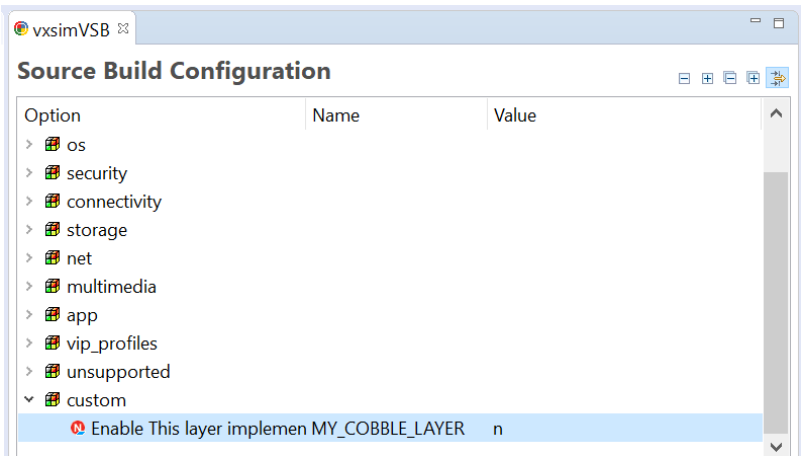
5. Use a host terminal to check that **cobbleLayer**, otherwise known as **MY_COBBLE_LAYER**, does exist.

```
C:\WindRiver\> wrenv -p vxworks-7
C:\WindRiver\> wrtool -data C:\WindRiver\workspace
Active workspace is 'C:\WindRiver\workspace'.
WrTool, version 4.1.0
Copyright (c) 2013 - 2014 Wind River Systems, Inc.


C:\WindRiver\> prj vsb info MY_COBBLE_LAYER
Name: MY_COBBLE_LAYER
[...]
[...]
Location: C:/WindRiver/vxworks-7/pkgs_v2/custom/cobbleLayer
```

6. Verify that your VSB has access to **MY_COBBLE_LAYER** by opening the VSB source build configuration in the project explorer.

   You can see **MY_COBBLE_LAYER** appearing as a component within the Source Build Configuration and the **custom** container.



7. Include **cobbleLayer** in your VSB.
   1. Right-click on the red component within **custom** and select **Add with Dependencies** to add **MY_COBBLE_LAYER** to the VSB.
   2. Observe **MY_COBBLE_LAYER** is now a green component.
   3. In the project explorer, right-click on the VSB and select **Rebuild Project**.
8. Modify your VIP to start the cobble application upon target startup.
   1. Locate the VIP related to your VSB in the project explorer.
   2. Locate the **usrAppInit.c** file within the project and open it for editing.
   3. Insert a call to the **cobble.c** progStart( ) function into the **usrAppInit.c** usrAppInit( ) function.

```
/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef  USER_APPL_INIT
    USER_APPL_INIT;     /* for backwards compatibility */
#endif

    /* TODO: add application specific code here */
    progStart();
    }
```

9. Rebuild the VIP and reboot the target from the rebuilt VIP. Check that the expected cobble tasks are running on the target from the VxWorks shell.

   The **cobble.c** file is a simple multi-tasking application that simulates a data collection system in which data is retrieved from an external source (for example, a device that generates interrupts as data comes in). The four cobble tasks are:

   - tCosmos, which simulates an interrupt service routine (ISR) and generates new data.
   - tSchlep, which collects the data.
   - tCrunch, which performs calculations on the data to obtain a result value.
   - tMonitor, which monitors the results and prints a warning message if they are outside of the safety range.

   > **Note:** If you do not add progStart( ) to **usrAppInit.c**, cobble does not start upon target reboot and **cobbleLayer** does not link into the kernel image. Rather, the layer is omitted by the VxWorks build system because there is no reference to the **cobble.c** application in the VxWorks BSP or application code. As a consequence, the VxWorks build system linker omits **cobbleLayer** during the link phase of the VIP build.

```
-> i

  NAME         ENTRY       TID    PRI   STATUS      PC       SP     ERRNO  DELAY
----------  ------------ -------- --- ---------- -------- -------- ------- -----
tIsr0       102678f0     3800da08   0 PEND       102714ee 380cff60       0     0
tJobTask    101a82b0     38049e08   0 PEND       102714ee 3813ff00       0     0
tExcTask    101a7ca0     103dfb40   0 PEND       102714ee 103e5d40       0     0
tLogTask    logTask      3804dd48   0 PEND       1026f073 3817feb0       0     0
tShell0     shellTask    38369d40   1 READY      1027a760 3841fc40       0     0
tErfTask    100ed8c0     38002e10  10 PEND       10271e1f 3802ff00       0     0
ipcom_tick> 1003a1b0     38437e48  20 PEND       102714ee 384dfed0       0     0
tVxdbgTask  100ffc70     3832b4b0  25 PEND       102714ee 3838ff20       0     0
tTcfEvents  1020f1f0     103f8380  49 PEND+T     102714ee 103f81f0       0    54
tTcf        1015a0b0     383ffb18  49 PEND       102714ee 3845f890       0     0
tTcf        1015a0b0     38433a10  49 PEND       102714ee 3849f8a0       0     0
tTcf        1015a0b0     38479908  49 PEND       102714ee 3861f8f0       0     0
tTcf        1015a0b0     3847d5c8  49 PEND       102714ee 3865fa40       0     0
tNet0       ipcomNetTask 38008b30  50 PEND       102714ee 3806fef0       0     0
tAioIoTask> aioIoTask    3811b440  50 PEND       10271e1f 381ffef0       0     0
tAioIoTask> aioIoTask    3811e870  50 PEND       10271e1f 3823fef0       0     0
ipcom_sysl> 10038c10     38086280  50 PEND       10271e1f 382bfe20       0     0
tNetConf    10075840     380a5ec8  50 PEND       102714ee 3834fcf0       0     0
tAnalysisA> cafe_event_> 103d9ca0  50 PEND       102714ee 103d9b30       0     0
tAioWait    aioWaitTask  38117f10  51 PEND       102714ee 381bfe80       0     0
tCosmos     cosmos       38438518 200 DELAY      10279812 3851ff80       0     1
tSchlep     schlep       38438948 220 PEND       102714ee 3855ff20       0     0
tMonitor    monitor      3843f900 230 READY      10149800 385dffb0       0     0
tCrunch     crunch       3843f5d0 240 READY      100b5ae0 3859ffd0       0     0
ipcom_egd   10037c10     38089998 255 READY      10037c80 382ffec0       0     0
value = 0 = 0x0
->
```


WNDRVR

3. EXAMPLE: MIGRATING AN RTP PROJECT INTO A LAYER

VxWorks is composed of both Wind River supplied and user cutomized layers. Layers typically contain kernel mode software components. However - layers can also encapsulate user mode Real Time Process (RTP) software components too.

Before You Begin

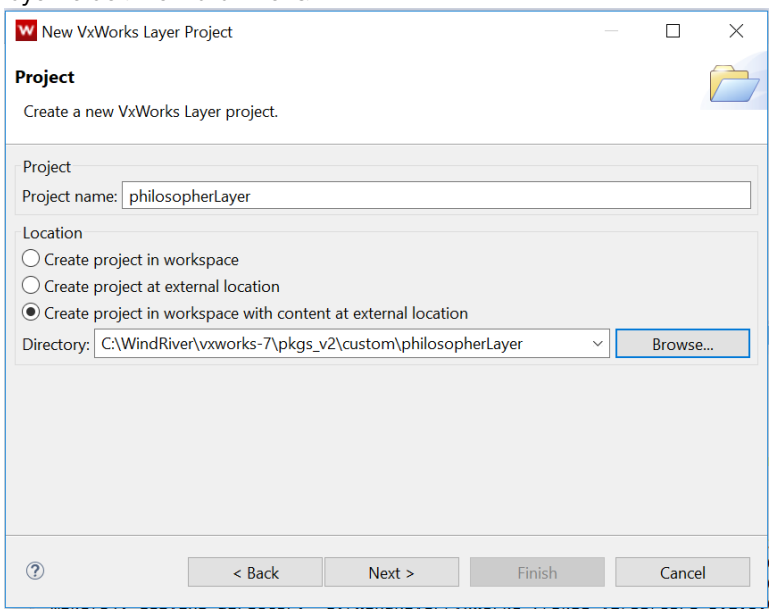
This example assumes you have already completed Example: Creating an Inside Custom Layer and Integrating it into a VSB on page 2. It also assumes the Workbench **Project Explorer** contains a Philosophers Example RTP Project.

About This Task

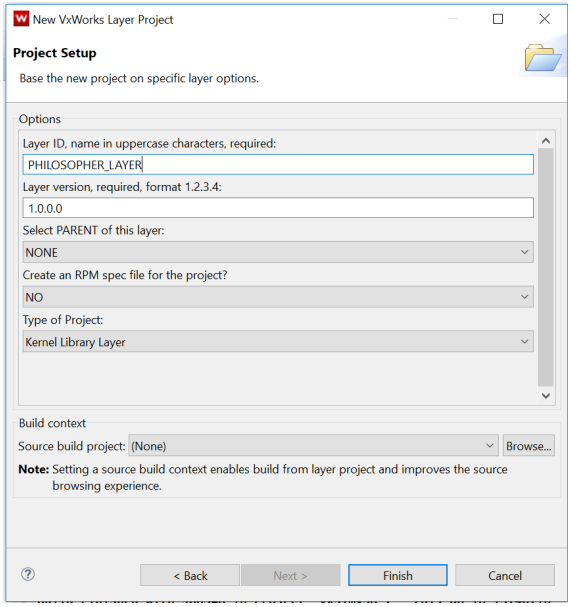
You will insert a pre-existing Workbench RTP Project into a VxWorks 7 layer. This allows the RTP to become part of a VxWorks 7 VSB that builds along with the VSB VxWorks 7 kernel libraries. You will build a VSB containing a philosophers RTP layer, boot the target with the resulting VIP and then run the philosophers RTP layer executable.

Procedure

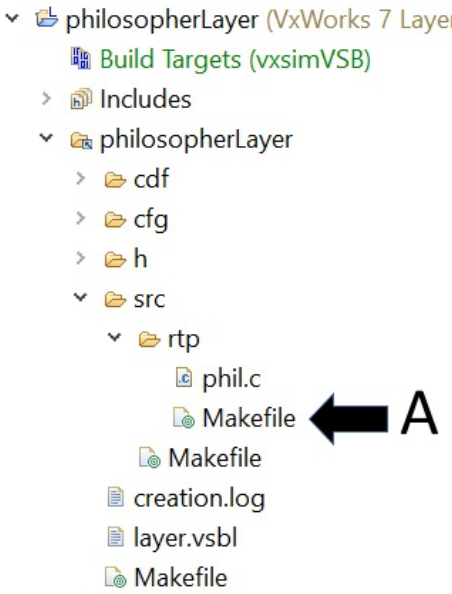
1. Create a folder for your new layer projects.
    1. Create a folder in the Wind River installation at *installDir* **\vxworks-7\pkgs_v2\custom**.
    2. Create a folder for your new custom layer project at *installDir* **\vxworks-7\pkgs_v2\custom\philosopherLayer**.
2. Create a VxWorks layer project in your new layer project folder.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. From the Build type drop down list select **Layer** then click **Next**.
    3. Give the Project the name **philosopherLayer**.
    4. Under the **Location** dialog select **Create project in workspace with content at external location** and **Browse** to the layer folder. Now click **Next**.



    5. Enter **PHILOSOPHER_LAYER** as the layer ID. Leave all other options as default and select **Finish**.



3. Populate **philosopherLayer** with **phil.c**.
    1. In the **Project Explorer**, navigate to **philosopherLayer**.
    2. Right-click on the **src** directory and select **New**, **Folder**. Name the new folder **rtp**.
    3. In the **Project Explorer**, expand the **philosophers** project.
    4. Right-click on the **phil.c** source file and select **Copy**.
    5. Highlight the **rtp** folder under **philosopherLayer** and select **Paste**.
4. Create the layer rtp **Makefile**.
    1. Highlight the **rtp** folder under **philosopherLayer** and right-click.
    2. Select **New**, **File** and name it **Makefile**.
    3. Verify that the **Makefile** exists in the **rtp** directory. Refer to the example, the **Makefile** is pointed to as A.



5. Prepare the layer rtp **Makefile** A to produce an RTP layer.
    1. Edit **Makefile** A and append a name for the RTP executable to the EXE declaration line.

> **Note:** This declares the file name of the RTP executable file that is produced as part of the VSB build.

2. Define *RTP_BASE_DIR*, which is the name of the original **philosophers** example directory.
3. Define *OBJS* which declares the filename of the compiled philosophers object file which is produced by the VSB build.
4. Include the RTP Makefile fragment **$(WIND_USR_MK)/rules.rtp.mk**.

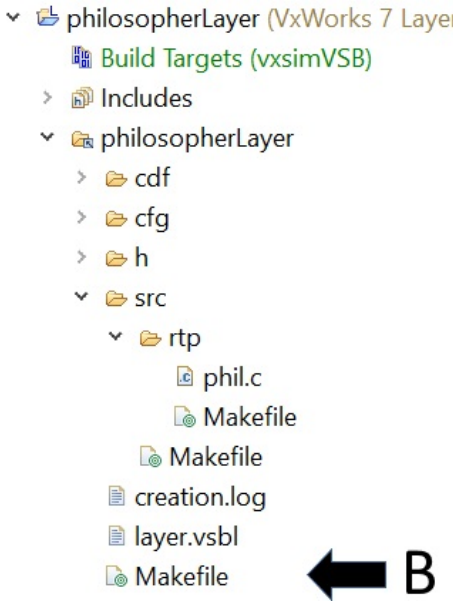
```
EXE = phil.vxe

RTP_BASE_DIR = philosophers

OBJS = phil.o

include $(WIND_USR_MK)/rules.rtp.mk
```

6. Identify the **philsopherLayer** root **Makefile**. Refer to the example, the **Makefile** is pointed to as B.



7. Prepare this general purpose **Makefile** B to produce an RTP layer.
   1. Add the following line to the end of the **Makefile**, before the include line:

```
POSTBUILD_RTP_DIRS = src/rtp
```

   2. Delete the contents of this **Makefile** so that only the following remains:

```
# Modify the following settings if your layer is being used to generate RTP executables in t
he VSB
# POSTBUILD_RTP_DIRS Post-build RTP source Directories; preferred directory name is rtp
POSTBUILD_RTP_DIRS = src/rtp


include $(WIND_KRNL_MK)/rules.layers.mk
```

8. Identify the **philosopherLayer** project meta-data file **layer.vsbl** in the root of the project. Modify the **layer.vsbl** file. Identify the line *VSB_REQUIRES* and add **RTP** required as follows:

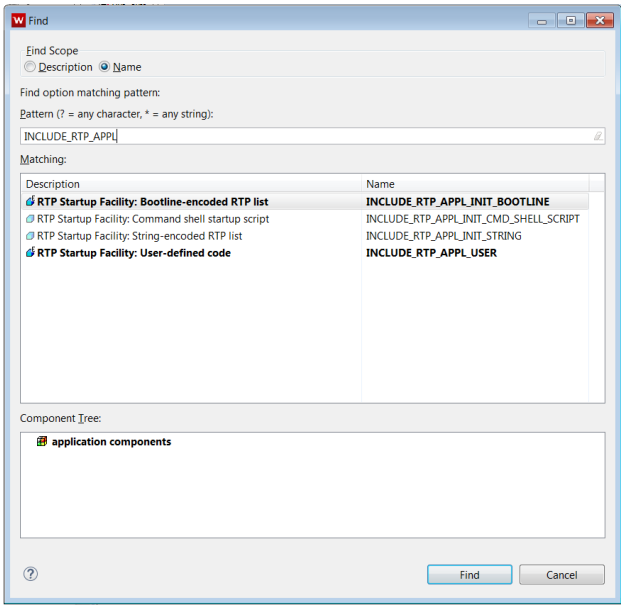
```
Layer PHILOSOPHER_LAYER {
SYNOPSIS This should be the synopsis for PHILOSOPHER_LAYER
VERSION 1.0.0.0
```

```
VSB_REQUIRES    RTP
LAYER_REQUIRES
SELECT
FEATURE_PROVIDES
OPTIONAL YES
DEFAULT YES
PARENT
DOC_BUILD NO
FASTBUILD YES
HELP This should be the descriptive help for PHILOSOPHER_LAYER
}
```

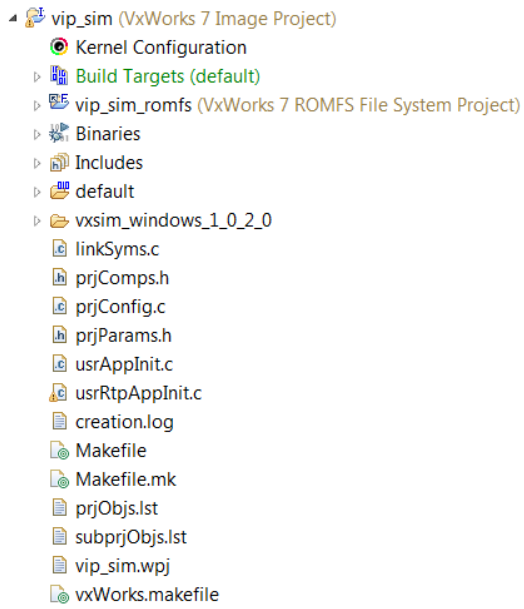
9. Rebuild the VSB.

> **Note:** The VSB build will also build **philosopherLayer**, generating a file called **phil.vxe**. Locate this file in your workspace inside the VSB project file system under **usr/root/llvm/bin/phil.vxe**.

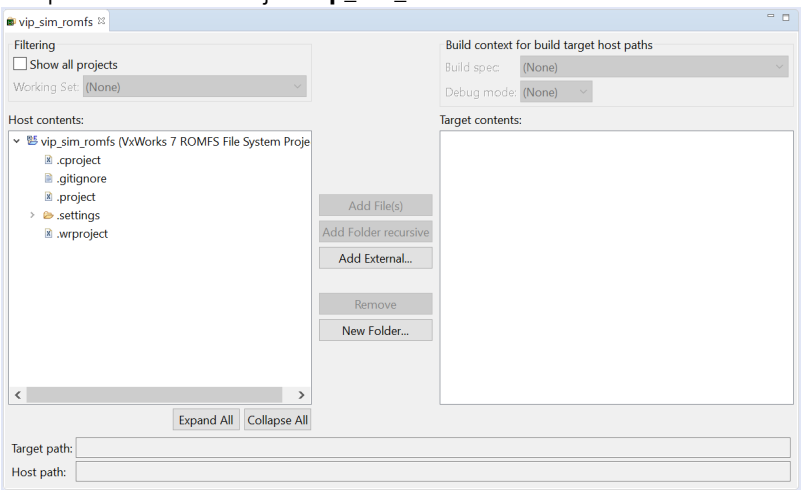
10. Prepare your VIP to start an RTP automatically at boot time.
    1. Open the VIP Kernel Configuration in the **Project Explorer**.
    2. Use CTRL-F in Kernel Configuration to locate **INCLUDE_RTP_APPL_USER**. Include this kernel component.
    3. Use CTRL-F in Kernel Configuration to locate **INCLUDE_ROMFS**. Include this kernel component.



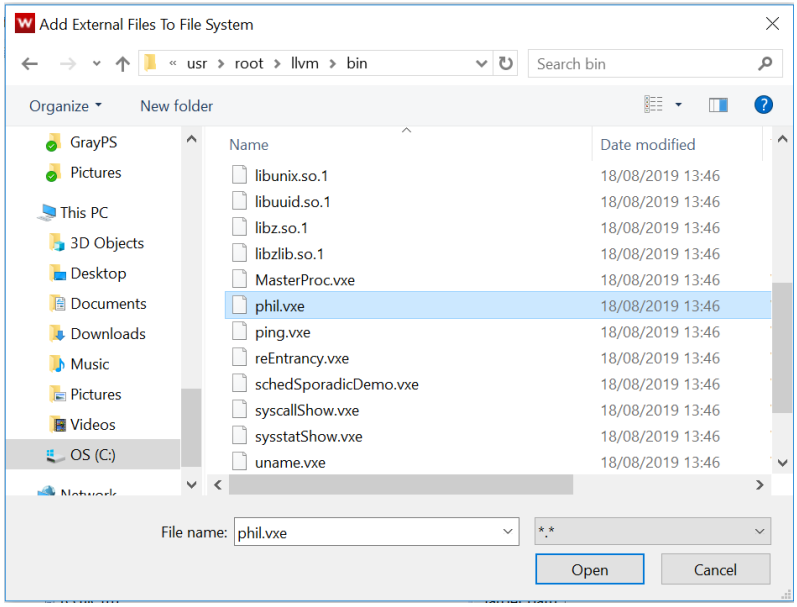
11. Add a **ROMFS File System** containing **philosopherLayer** to your VIP.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. From the Build type drop down list select **ROMFS File System** then click **Next**.
    3. Name the Project **vip_sim_romfs** and click **Finish**. If necessary, drag **vip_sim_romfs** into the VIP project in the **Project Explorer** so that the ROMFS project builds as part of the the VIP.



4. Open the ROMFS Project **vip_sim_romfs** and click the **VxWorks ROMFS File System Contents**.



5. Click the **Add External** button. Navigate to the **phil.vxe** file, generated by the VSB build and contained inside the VSB project file system under **usr/root/llvm/bin/phil.vxe** and click **Open**.



12. Rebuild the VIP Project. The ROMFS project also builds, containing **phil.vxe**.
13. Modify VIP startup to cause **philsopherLayer** to run at target boot time.
    1. Locate the VIP Project file **usrRtpAppInit.c** .
    2. Modify **usrRtpAppInit.c** to spawn philosophers at target boot time.

> Note: The path to **phil.vxe** is the file path in the target's ROMFS file system.

```c
#include <stdio.h>
#include <stdlib.h>
#include <taskLib.h>
#include <sysLib.h>
#include <rtpLib.h>

/*******************************************************************************
*
* usrRtpAppInit - initialize the users RTP applications.
*/

void usrRtpAppInit (void)
    {
    /* TODO - add your own application launch code here */
    char * vxeName = "/romfs/phil.vxe";
    char * argv[5];
    RTP_ID rtpId = NULL;

    /* set the application's arguments */

    argv[0] = vxeName;
    argv[1] = NULL;
    argv[2] = NULL;
    argv[3] = NULL;
    argv[4] = NULL;

    /* Spawn the RTP. No environment variables are passed */
    printf("usrRtpAppinit()\n");
    if ((rtpId = rtpSpawn (vxeName, argv, NULL, 220, 0x10000, 0, 0)) == RTP_ID_ERROR)
    {
    printErr ("Impossible to start myVxApp application (errno = %#x)",
    errno);
    }
    }
```

14. Rebuild the VIP and reboot the target. The **philosopherLayer** will run on target startup.

> Note: If the **philosopherLayer** does not run, ensure that the **INCLUDE_RTP_INCLUDE_APPL** component is included in the VIP, and also ensure that we have modified usrRtpAppInit( ) exactly as above.

```
 _________           __________
 \77777777\          /77777777/
  \77777777\        /77777777/
   \77777777\      /77777777/
    \77777777\    /77777777/
     \77777777\   \777777/
      \77777777\   \77777/              VxWorks 7
       \77777777\   \777/
        \77777777\   \7/     Core Kernel version: 1.0.10.2
         \77777777\   -      Build date: Jun 24 2015 16:08:51
          \77777777\
           \7777777/         Copyright Wind River Systems, Inc.
            \77777/   -                  1984-2015
             \777/   /7\
              \7/   /777\
               -   -------

              Board: SIMNT board
     OS Memory Size: 511MB
   ED&R Policy Mode: Deployed
        Debug Agent: Started (always) and modified by Me

Loading symbol table from host:C:/NoScan/WindRiver/workspace/vip_sim/default/vxWorks.sym ...done

usrRtpAppinit()
->
Resume the RTP's initial task to end the demo.


Running claim-based solution.

Philosopher 1 is thinking...
Philosopher 2 is thinking...
Philosopher 3 is thinking...
Philosopher 4 is thinking...
Philosopher 5 is thinking...
Philosopher 1 wants to eat.
```